

Advanced Tools for Complex Network Analysis

CNET 5052, Spring 2026

Prof. Brennan Klein and Dr. Milo Trujillo

Assignment 4, due by Tuesday, April 7, 2026 (a.o.e.)

Please submit a .pdf of your answers via Canvas, including a url to your .ipynb notebook in your fork of our class repository. Please name your Canvas submission as Lastname.pdf, (e.g. Klein.pdf). If you need help with LaTeX or GitHub, please visit us during office hours or post questions to the Slack channel.

AI Tags. Each question below has a tag indicating what kind of AI usage is appropriate, if at all:

- ✧ [AI-FORBIDDEN](#) ✧ : Do not use generative AI for this. We want your own thinking and voice.
- ✧ [AI-LIMITED](#) ✧ : You may use AI for debugging help, documentation help, or short clarifications. Do not paste in complete solutions. You must still understand and be able to explain your work.
- ✧ [AI-PERMITTED](#) ✧ : You may use AI as a helper (including code suggestions), but you must cite it, verify by running and testing, and write your own explanations.
- ✧ [AI-ENCOURAGED](#) ✧ : You are encouraged to use AI as a learning partner, if helpful. These parts are meant to build programming confidence and grasp.

In this course, the use of AI tools is experimental and intended to enhance your work and learning. Please use this privilege wisely, as we reserve the right to revoke this policy.

1. Network reconstruction (25 points).

✧ [AI-LIMITED](#) ✧

In this problem, you will simulate dynamics on a known graph, reconstruct the graph from the observed time series, and think carefully about how to evaluate reconstruction quality. This question builds directly on our recent discussion of network reconstruction as an inverse problem: a graph produces data, and we ask whether the data are informative enough to recover the hidden graph. Recall from `class_08` that it is useful to distinguish between an *edge-score matrix* S , whose entries reflect evidence for edges, and a thresholded *reconstructed adjacency* $\hat{A}$.

- Using `netrd`, simulate a dynamical process on a known graph G with $n = 64$ nodes—pick any (connected) network you’d like—for $T = 1000$ time steps, producing matrix $X \in \mathbb{R}^{T \times n}$. Then repeat this 20 times on the *same* underlying graph, using different random seeds or noise realizations each time. Describe your choice of graph and dynamics. Visualize one of the time series matrices.
- Using a network reconstruction method of your choosing (from `netrd` will likely be most straightforward), reconstruct each time series you created into a reconstructed network R_i . Store these 20 networks in a list.

- (c) Note that `netrd` can incorporate thresholding into the reconstruction process directly. `R = reconstructor.fit(TS, threshold_type='degree', avg_k=15)`, for example, will threshold the weighted reconstruction such that the final output will have average degree of 15. Re-do step (b) above, and threshold your reconstructed networks based on degree, setting `avg_k` equal to the average degree of your original network G .
- (d) How would you go about evaluating the “correctness” of your reconstructed networks? Introduce a measure, $D(G, R)$, that takes two graphs as input—your original network G and a reconstructed (and thresholded) network R —and outputs a single scalar value. Use this function to quantify how “close” each of your 20 reconstructed networks are to the original graph. Create a bar plot of these values with 20 bars, corresponding to the $D(G, R_i)$ of each of your reconstructed networks, R_i .
- (e) Use your dissimilarity measure D to compute $D(R_i, R_j)$ between each pair of the 20 reconstructed networks you have created. Put these into a distance matrix d , such that d_{ij} corresponds to how different reconstructed networks R_i and R_j are from one another. Visualize this matrix using `plt.imshow()`.
- (f) What would it mean if all of your reconstructed networks are all very *close* or *similar* to one another? What if they were far/dissimilar? How might the choice of dynamical process, original network G , reconstruction algorithm, and/or thresholding approach all influence how effective or ineffective a given reconstruction technique is? Note that this question is designed to be quite open-ended. There is no single “correct” answer! We just want to see you think through this complicated question.

2. Big data (25 points).

✧ AI-LIMITED ✧

We are going to continue playing with *Bloom Filters*. Probabilistic data structures like Bloom Filters and HyperLogLog are most useful when working with terabytes of data, so we will create a somewhat contrived toy problem to demonstrate their utility with smaller data. We will begin by creating a bloom filter as pre-cache to an enormous adjacency list.

- (a) Create a bloom filter for storing edge data. Rather than using strings as keys, as in class, you will use pairs of integers as keys, representing edges. Your bloom filter should take two arguments of n and ϵ , representing approximately how many keys will be added (here, the number of edges on a graph), and your desired error rate (1%, 5%, etcetera).
- (b) Create an Erdős-Rényi graph with 1,000 nodes and a 5% edge probability. Add the edges to your bloom filter. This toy graph will represent a much larger graph, such as the follow relations of all Twitter users. Let us assume that checking for edge existence in the full graph takes one second, to keep our math simple.
For all possible edges, how long would it take to check for their existence in the complete graph? How long would it take to check if those edges exist if you *first*

reference your bloom filter? How does this answer change as you adjust ϵ from 0.01 to 0.05 to 0.1?

- (c) Bloom Filters are of length m , where m is based on the desired error rate and the number of elements that will be stored:

$$m = -\frac{n \ln \epsilon}{(\ln 2)^2}$$

However, the total number of elements cannot always be known upfront. We will now explore how sensitive Bloom Filters are to mis-estimates of n . We will begin with bloom filters where m is sized for the ER graph in part 2. What if you add 1.5 times as many edges as anticipated? Twice as many? Three times as many? When and how does the Bloom Filter “break down”? What happens to our anticipated lookup times when using a bloom filter as pre-cache versus consulting the original graph for edge existence directly? Create a line plot with number of elements on the x-axis, and a value of your choosing on the y-axis that best demonstrates this issue, with three lines for $\epsilon \in [0.01, 0.05, 0.1]$. We would like a written and visual explanation of the issue.

3. ABMs: exploration, exploitation, and network structure (50 points).

✧ AI-PERMITTED ✧

In this problem, you will study how network structure affects collective problem solving on a rugged landscape. This question is inspired by Lazer and Friedman (2007)¹, who used an agent-based model on an NK landscape to study how communication networks shape exploration and exploitation.

- (a) Read Lazer & Friedman (2007).
 - i. How do they operationalize *exploration* and *exploitation*?
 - ii. What is an *NK landscape*?
 - iii. Describe Figure 4. What quantity is plotted over time, what two network structures are compared, and what is the main qualitative result?
 - iv. Describe Figure 5. How does it help explain the result in Figure 4?
- (b) We will now begin a multi-step process of replicating Figure 4 from Lazer & Friedman (2007). We have provided you with a function that generates an NK landscape and returns a fitness function on binary strings of length N . We will also provide a helper function that generates a one-bit mutation of a binary string. Each agent holds one current solution, represented as a binary string. At each time step, every agent updates according to the following three-step rule:
 - The agent checks the current solutions of its neighbors.
 - If at least one neighbor has strictly higher fitness, the agent copies the highest-fitness neighbor.

¹ <https://ndg.asc.upenn.edu/wp-content/uploads/2016/04/Lazer-Friedman-2007-ASQ.pdf>

- Otherwise, the agent tests one random one-bit mutation of its current solution. If the mutation has higher fitness, the agent adopts it. If not, the agent stays where it is.

Notes:

- Use *synchronous* updating: all agents should make decisions based on the states at time t , *and then* all updates should be applied simultaneously to produce time $t + 1$.
 - When comparing two different network structures, you should hold fixed the NK landscape and the initial agent solutions within each replicate so that the only thing changing is the network.
 - In this problem, one *replicate* means one NK landscape together with one initial population of agent solutions.
 - If multiple neighbors tie for the highest fitness and all are strictly better than the focal agent, break the tie uniformly at random.
 - Record all quantities at times $t = 0, 1, \dots, 100$, where $t = 0$ denotes the initial condition.
- (c) Before running the full model, write code that takes a population of agent solutions and computes the following three quantities:
- i. the average fitness in the population,
 - ii. the best fitness in the population,
 - iii. the number of unique solutions in the population.
- Use this code to verify that if the path graph and complete graph begin from the same initial solutions, then these three quantities are identical at time $t = 0$.
- (d) Implement this agent-based model on two networks with the same number of agents: a path graph and a complete graph. Use 100 agents, $N = 20$, and $K = 5$. Start each agent at a random binary string. For one example simulation on each network, run the model for 100 time steps and plot:
- i. the average fitness in the population over time,
 - ii. the best fitness in the population over time,
 - iii. the number of unique solutions in the population over time.

Briefly compare the two single-run trajectories. In particular, comment on whether one network seems to improve more quickly at first, and whether one network seems to preserve diversity for longer.

- (e) A single run is not enough. Repeat the experiment from part (d) for at least 100 independent replicates. In each replicate, the path graph and complete graph should use the same NK landscape and the same initial agent solutions. Then create a Figure-4-style plot showing the mean *average fitness in the population* over time for the path graph and the complete graph, with uncertainty bands of your choice. In a second plot, show the mean number of unique solutions over time for the two networks. Based on your results, which network appears better in the short run? Which appears better in the long run? Explain your answer in terms of exploration, exploitation, and diversity.

- (f) Now study how this comparison depends on the *ruggedness* of the landscape. Repeat part (e) for $K \in \{0, 1, 5, 10\}$, holding N fixed. Define the *long-run performance* of a network to be its mean average population fitness at time $t = 100$. Create one figure that summarizes how the long-run performance difference between the complete graph and the path graph changes as K increases. Then explain, in words, why changing K changes the answer.
- (g) Briefly reflect on simulation design. Why is it important here to run many replicates, hold the landscape fixed when comparing networks within a replicate, and measure solution diversity in addition to fitness?
- (h) Based on your replication (congrats!), recommend one research question that could build on this foundational result. It can be as simple as one sentence, e.g. “what if they changed X or Y etc...?” or as complicated as a fully-developed research question. *Opportunity for up to +5 extra credit points.*

A note on how to approach this problem:

This is a long, multi-step question, but it becomes much more manageable if you think about it piece by piece. First: what are the core objects in this model? We’ve got a *landscape*, which tells us how good or bad any candidate binary-string solution is. There are *agents*, each of whom holds one current solution at any time. There is a *graph*, which tells you who talks to whom. The graph is the communication structure; the population of agent solutions is the current state living on top of that structure.

So what do you need first? You need a landscape. We have provided you with a function to *generate an NK landscape*. Look carefully at its inputs. What do N and K mean? What does the function return? How will we use the returned fitness function to score one binary string? Next, once you have a landscape, you need to initialize the agents. How many agents are there? What does one agent’s state look like? How will you represent the full population of agents at time $t = 0$? A list? A dictionary? Etc.? Since each agent begins with a binary-string solution, what kind of Python object makes sense for storing the whole population?

Then, think carefully about **one round** of the game. What happens in one update step? For a given agent, what information do they need? They need to know their own current solution, the solutions of their neighbors, and the fitnesses of those solutions. If a better neighbor exists, what should happen? If no better neighbor exists, what should happen? Say it aloud and think through what the process is, refer to the original article too.

What does it mean to run the model for many rounds? If you simulate 100 time steps, what do you want to keep track of after each step? At minimum, this question asks you to track average fitness, best fitness, and the number of unique solutions in the population.

Once you can run one simulation, you are not done. A single run can be noisy. The next step is to repeat the same experiment many times. But be careful: within each replicate, if you want to compare two network structures fairly, you should hold fixed

the NK landscape and the initial population of solutions. The only thing that should change is the graph.

Finally, when you analyze your results, think mechanistically. Do not just report which network “wins.” Ask why! What is happening to the diversity of solutions? Does one network spread good ideas quickly but collapse to a small number of solutions? Does another preserve diversity for longer? How does this connect to exploration and exploitation? In other words, work in layers: landscape, initial population, one update step, many update steps, repeated replicates, and then interpretation. If you build the simulation in that order, the question is much more approachable.

Below is the helper function we are providing for NK landscape generation. You may use it directly in your solution.

```
import numpy as np

# =====
# PART 1. NK LANDSCAPE GENERATOR
# =====

def generate_nk_landscape(N=20, K=5, seed=None, apply_paper_skew=True):
    """
    Create one NK landscape and return a fitness function on binary
    ↪ strings.

    This is the "environment" or "problem space" that agents search over.
    It does NOT implement the agent-based model itself. It only defines
    how good or bad each binary-string solution is.

    -----
    BIG PICTURE
    -----
    In an NK landscape:
        - N is the length of the binary string.
        - K controls ruggedness.

    Each solution is a binary string such as:
        (0, 1, 1, 0, 0, 1, ...)

    The score of a solution is built from N local contributions.
    Each local contribution depends on:
        - the bit at one locus i
        - and K other loci that interact with it

    As K increases, the landscape usually becomes more rugged:
        - more local optima
        - more path dependence
        - more chance of getting stuck

    -----
    CONNECTION TO LAZER & FRIEDMAN (2007)
    -----
    This follows the standard NK setup described in Appendix A of the
    ↪ paper.
```

The paper:

- uses an NK landscape as the search environment,
- normalizes performance by the maximum of the landscape,
- and discusses an r^8 monotonic transformation in Appendix A.

This function:

1. builds the local dependency structure,
2. creates random contribution tables,
3. computes the score of every possible bitstring,
4. normalizes by the exact global maximum,
5. optionally applies the paper-style r^8 transformation.

IMPORTANT DISTINCTION

This function only builds the landscape. It does NOT: create agents,
 ↪ create a graph, etc.

Those belong to the ABM code that you will write.

PARAMETERS

N : int

Length of the bitstring.

K : int

Number of partner loci that affect each locus.
 Must satisfy $K < N$.

seed : int or None

Random seed for reproducibility.

apply_paper_skew : bool

If True, apply the paper's monotonic r^8 transformation after
 normalization by the global maximum.

RETURNS

landscape : dict

Contains:

- "fitness_fn": callable that scores a bitstring
- "N", "K"
- "partners": dependency structure
- "tables": local NK contribution tables
- "raw_scores": raw NK scores over all 2^N states
- "normalized_scores": raw_scores / max(raw_scores)
- "scores": final scores after optional r^8 transform
- "global_max_raw": exact maximum raw score
- "global_best_bits": one bitstring attaining the max

COMPUTATIONAL NOTE

This version explicitly enumerates all 2^N states.

```

That is feasible for N = 20:
     $2^{20} = 1,048,576$  states

That is one reason N = 20 is still manageable here.
    """

    if K >= N:
        raise ValueError("K must be less than N.")

    rng = np.random.default_rng(seed)

    def bits_to_int(bits):
        """
        Convert a bitstring into an integer index.

        Example:
            (0,0,0) -> 0
            (0,0,1) -> 1
            (0,1,0) -> 2
            ...
        """
        x = 0
        for b in bits:
            x = (x << 1) | int(b)
        return x

    def int_to_bits(x, N):
        """
        Convert an integer back into an N-bit tuple.
        """
        return tuple((x >> shift) & 1 for shift in range(N - 1, -1, -1))

    # -----
    # Step 1: choose K partner loci for each locus i
    # -----
    partners = []
    for i in range(N):
        others = [j for j in range(N) if j != i]
        chosen = tuple(rng.choice(others, size=K, replace=False))
        partners.append(chosen)

    # -----
    # Step 2: create random local contribution tables
    # Each locus gets a table with 2^(K+1) entries
    # -----
    tables = []
    for i in range(N):
        table = rng.random(2 ** (K + 1))
        tables.append(table)

    # -----
    # Step 3: enumerate all 2^N possible solutions
    # -----
    num_states = 2 ** N
    states = np.arange(num_states, dtype=np.uint32)[: , None]
    shifts = np.arange(N - 1, -1, -1, dtype=np.uint32)
    bit_matrix = ((states >> shifts) & 1).astype(np.uint8)

```



```

# -----
# Step 4: compute raw NK score for every state
# -----
raw_scores = np.zeros(num_states, dtype=np.float64)

for i in range(N):
    # The local contribution for locus i depends on:
    #   - bit i itself
    #   - the K partner bits chosen for i
    idxs = (i,) + partners[i]
    local_bits = bit_matrix[:, idxs]

    # Convert each local bit-pattern into an integer lookup index
    local_powers = (1 << np.arange(K, -1, -1)).astype(np.uint32)
    table_indices = (local_bits * local_powers).sum(axis=1)

    raw_scores += tables[i][table_indices]

raw_scores /= N

# -----
# Step 5: normalize by the exact global maximum
# -----
global_max_raw = float(raw_scores.max())
normalized_scores = raw_scores / global_max_raw

# -----
# Step 6: optional paper-style skew transformation
# -----
if apply_paper_skew:
    scores = normalized_scores ** 8
else:
    scores = normalized_scores.copy()

argmax_state = int(raw_scores.argmax())
global_best_bits = int_to_bits(argmax_state, N)

def fitness_fn(bits):
    """
    Return the final fitness score of one bitstring.
    """
    idx = bits_to_int(bits)
    return float(scores[idx])

return {
    "fitness_fn": fitness_fn,
    "N": N,
    "K": K,
    "partners": partners,
    "tables": tables,
    "raw_scores": raw_scores,
    "normalized_scores": normalized_scores,
    "scores": scores,
    "global_max_raw": global_max_raw,
    "global_best_bits": global_best_bits,
}

```